

Graded Where-Clauses

Non-crisp truth values as the syntax of non-determinism resolution
in the rho calculus, with quantum and probabilistic-logic instances

L. G. Meredith
F1R3FLY.io

F1R3FLY.io working note — August 2026

Abstract

The rho calculus is silent about how the non-determinism of a race is resolved. An implementation must supply that mechanism, and different mechanisms give different executions of the same program: RSpace supplies the physical races of cores on a CPU, and one may equally supply the Born rule. We take this seriously as a design principle and ask what syntax a programmer needs in order to *arrange* a resolution mechanism rather than merely inherit one.

We argue, first, that no repair of full abstraction over rho contexts can expose the additional structure a quantum resolver supplies, because no rho context says anything about how non-determinism is resolved; the mismatch is not in the class of contexts but in the type of observation. We then show that unmodified rho, read as a sum over its possible futures, already interferes — but only on structurally indistinguishable racers, and only constructively, because the message left behind by a race is a which-path record.

The design that follows is a single change: the **where** clause of the for-comprehension is valued in a commutative semiring $\mathbb{V}$ rather than in the Booleans, and is evaluated *per candidate match* rather than per communication. Everything else is unchanged. We show that (i) weight maps in the sense of the weighted-GSLT programme are exactly the linear normal forms of graded clauses, so the partition discipline and the update-function machinery both disappear; (ii) the join operator $\&$ is already a recombiner, so the which-path erasure that interference requires needs no new term former; and (iii) the graded clause on an m -fold join over m data *is* a scattering matrix, with the amplitude of the symmetric outcome given by its permanent. Boolean $\mathbb{V}$ recovers rholang 1.4 exactly; $\mathbb{R}_{\geq 0}$ gives stochastic execution; $\mathbb{C}$ gives interference, including cancellation of an operationally reachable outcome.

As a worked example we build Shor’s algorithm in the fragment, with every number computed by a simulator that enumerates candidates rather than multiplying gate matrices. Universality is easy and modular exponentiation is ordinary rholang, but the exercise turns up a hazard rather than a triumph: with clauses unconstrained, renormalising at observation is post-selection, and the fragment computes PP. We therefore state the isometry condition that contains it, and observe that the fragment on which it is decidable is exactly the fragment a compiler to circuits can accept.

Finally we observe that the same slot accepts truth values in the sense of Goertzel’s Probabilistic Logic Networks. Valuing where-clauses by PLN inference makes MeTTaIL’s execution engine and its reasoning engine the same loop: what fires next is decided by what the system currently believes. We give the resolution algebra this requires, and are explicit about which part of PLN’s truth-value arithmetic supplies values and which part would have to be weakened to supply the algebra.

Contents

1 Introduction

3

1.1	Where the choice is	3
1.2	The programme, in two stages	3
1.3	Contributions	3
1.4	What this note does not do	4
2	Why context-based full abstraction cannot help	4
2.1	An asymmetry between two translations	4
2.2	The door that stays open	5
2.3	Separation as the correctness frame	5
3	What unmodified rho already gives	6
3.1	The race equation	6
3.2	The residual is a which-path record	6
3.3	The normalisation fork	7
4	Two requirements, derived	7
4.1	Reconvergence must be counted over causal histories	7
4.2	The two syntactic requirements	8
5	The design	8
5.1	Resolution algebras	8
5.2	Two sorts of formula	9
5.3	Syntax	9
5.4	Per-match evaluation	9
5.5	The graded modality is a transfer operator	10
6	Formal semantics	10
6.1	Terms	10
6.2	The state space	11
6.3	Readings	11
7	Weight maps are a normal form	12
8	The join is the recombiner	13
8.1	Permutations of a join are the interfering pair	13
8.2	The minimal interferometer	14
9	A worked example: Shor's algorithm	14
9.1	A qubit and a gate	14
9.2	Composition, and where the circuit's wires went	15
9.3	The classical part is free	15
9.4	The instance, assembled	15
9.5	The hazard: unnormalised clauses buy post-selection	16
9.6	What the example shows, and what it does not	17
10	PLN-valued where-clauses	17
10.1	Why this slot invites PLN	17
10.2	PLN truth values, briefly	17
10.3	The resolution algebra, honestly	18
10.4	Three regimes, one syntax	18
11	Implementation	19

12 Open threads	20
13 Conclusion	20

1 Introduction

1.1 Where the choice is

A rho program does not determine its own execution. Write

$$x!(Q_1) \mid \text{for}(y \Leftarrow x)P \mid x!(Q_2)$$

and the operational semantics permits two reductions. It does not say which one happens, and it does not say with what tendency. This is not an oversight; it is the defining feature of a mobile process calculus, and it is what makes the calculus a specification of interaction rather than of a machine.

An implementation must close the gap. RSpace closes it by presenting the parallel fragment as a store from channels to polarity-homogeneous bags and letting the physical race between cores decide which produce is zipped with which consume. That is a resolution mechanism, and it is a choice — in the precise sense that the family of admissible pairings, indexed by contended channels, has a section only when something supplies one.

The observation this note develops is that the resolution mechanism is a *parameter*, and that other values of the parameter are available. In particular, quantum mechanics is a resolution mechanism: it supplies amplitudes for the branches of a race and the Born rule for turning them into outcomes. Instantiating rho’s parameter with that mechanism is a well-posed thing to do, and the question this note answers is what syntax a programmer needs in order to control it.

1.2 The programme, in two stages

The work proceeds in two stages, and it is worth separating them sharply because they have different success criteria.

Stage one exposes the additional structure. A quantum resolver distinguishes programs that classical observation identifies, because it permits interference. That is a failure of full abstraction — the denotation is finer than contextual equivalence — and the failure is the feature. The set of pairs that classical observation identifies and the quantum denotation separates is the specification document for stage two.

Stage two gives that structure syntactic expression, so that a programmer can write into the gap on purpose. Aaronson’s summary of quantum programming is apt here: most of the discipline consists of arranging for interference to do something useful. A syntax for arranging interference is therefore a syntax for exploring what quantum advantage is available, and having one is a precondition for a verdict rather than a consequence of one.

The weighted-GSLT programme proposed *weight maps* for stage two: a table, carried alongside the term, from spatial-behavioural formulae refining the left-hand side of a rewrite rule to weights. This note argues that the weight map had the right *language* and the wrong *location*, and that relocating it into the for-comprehension’s **where** clause — by the simple device of letting formulae take non-crisp truth values — both subsumes it and repairs three separate defects at once.

1.3 Contributions

1. **A negative result about full abstraction (§2).** Enlarging the class of rho contexts cannot expose interference, because contexts can *create* apertures but cannot *read* them.

The mismatch is in the observation type, not the context class, and the correct frame is a *separation* between resolvers rather than a full-abstraction theorem between terms. We give the replacement correctness condition: one-sided support adequacy, whose failure to be two-sided is the interference.

2. **A sharp stage-one verdict** (§3). Under the race equation with uniform weights, unmodified rho interferes exactly when the racing messages are structurally congruent, and then only constructively, because the surviving message is a which-path record. The phenomenon is a permanent; it is BosonSampling-shaped, hence plausibly hard and definitely not universal.
3. **The design** (§§5–6). Value the **where** clause in a commutative semiring and evaluate it per candidate match. Boolean values recover rholang 1.4 on the nose (Theorem 6.4).
4. **Weight maps are a normal form** (§7, Theorem 7.2). Every weight map is a graded clause of the shape $\bigoplus_i v_i \otimes [\chi_i]$, and the partition discipline exists solely to make that shape single-valued — a requirement a formula satisfies for free. Dynamic weight maps become formulae mentioning the current term, so the update function disappears too (Proposition 7.4).
5. **The join is the recombiner** (§8, Theorem 8.2). Which-path erasure — the half of the problem the weight map never addressed — requires no new term former. A join $\&$ over m patterns on a contended channel has $m!$ candidate matches whose outcomes coincide precisely when the continuation is symmetric in the bound names, and the graded clause supplies the $m!$ amplitudes. The clause is a scattering matrix and the symmetric outcome carries its permanent.
6. **Shor’s algorithm, and an overshoot** (§9). The fragment is universal for quantum circuits (Lemmas 9.2–9.4), the circuit’s DAG is the term’s causal order (Proposition 9.5), and the classical part costs nothing (Corollary 9.7). But with clauses unconstrained the fragment decides PP by post-selection (Proposition 9.9), so the isometry condition of Definition 9.10 is a containment requirement rather than a convenience.
7. **PLN-valued clauses** (§10). The same slot accepts Probabilistic Logic Network truth values, making inference and execution one loop. We give the algebra honestly, including which of PLN’s operations supply values and which would need weakening to supply a semiring.

1.4 What this note does not do

It does not prove a quantum advantage; it builds the instrument with which one could be sought. It does not settle the normalisation question raised in §3.3, though it makes the fork explicit and recommends a branch. It does not treat replication or the full recursive fragment beyond noting where the budget must bite. And it does not attempt a categorical account of the construction, which we expect to be a Kleisli-style story over the semiring’s free-module monad and which we leave to a companion.

2 Why context-based full abstraction cannot help

2.1 An asymmetry between two translations

When one encodes the π -calculus into rho, a natural worry about full abstraction arises and a natural repair is available: rho contexts see strictly more than π contexts do, so demand preservation only against the images of π contexts. The repair is legitimate because the discrepancy it manages — rho’s ability to inspect the structure of a name — lies on an axis about which

both calculi are silent in the same way, namely how a race is resolved. Restricting the context class is orthogonal to resolution, so it costs nothing on that axis.

The translation from rho into quantum mechanics is not like this. Quantum mechanics makes a definite commitment about resolution: amplitudes compose linearly and outcomes obey the Born rule. The discrepancy is therefore *on* the axis about which rho is silent. No restriction or enlargement of the class of rho contexts can address it, for the simple reason that a rho context is itself a rho term, whose own races are resolved by the same unspecified mechanism. Composing silence with silence yields silence.

Principle 2.1 (Contexts create apertures; they do not read them). A rho context can change which races exist — by adding contention, by consuming a contender, by installing a new receive. It cannot observe how any race is resolved. Consequently no property of the form “for all rho contexts $C[-]$, ...” can be sensitive to the resolution mechanism.

This is the precise reason a full-abstraction theorem is not the right correctness statement for the present setting, and the reason is stronger than the usual observation that interference makes the denotation too fine. It is not that we have too few contexts. It is that the discriminating power we want is not of the kind contexts have.

2.2 The door that stays open

The enterprise is not thereby vacuous, and it is worth being careful about why. A context cannot see a phase. But a context can be an *interferometer*: an arrangement in which a difference of phase becomes a difference in the *support* of the outcomes. This is how every physical quantum experiment reports its result — not by observing an amplitude but by a detector that never fires. §8 exhibits such a context in three lines of rholang.

What this does not do is rescue full abstraction, because the discrimination it achieves is between two *resolvers* at a fixed term, not between two terms at a fixed resolver.

2.3 Separation as the correctness frame

The right object of study is therefore the map

$$\text{Resolvers} \times \text{Terms} \longrightarrow \text{Distributions over normal forms},$$

in which the classical RSpace semantics and the Born semantics are two points of the first coordinate. Quantum advantage is the assertion that the image of the quantum resolver is not contained in the image of the classical family — a separation, in the shape of a Bell argument, rather than a full-abstraction theorem.

What survives as a checkable correctness obligation is one-sided.

Definition 2.2 (Support adequacy and the interference deficit). Let $\text{Reach}(P) \subseteq \text{NF}$ be the set of normal forms operationally reachable from P , and let $\llbracket P \rrbracket$ be the graded denotation of §6. The denotation is *support-adequate* when

$$\text{supp}(\llbracket P \rrbracket) \subseteq \text{Reach}(P).$$

The *interference deficit* of P is $\mathcal{D}(P) := \text{Reach}(P) \setminus \text{supp}(\llbracket P \rrbracket)$.

Support adequacy says the interpretation invents nothing the operational semantics forbids. The converse inclusion is exactly what destructive interference violates: a normal form that some derivation reaches, whose amplitudes cancel. So the natural two-sided condition is not merely unproven, it is *false precisely where the interesting physics is*, and its failure is the observable. We will return to $\mathcal{D}$ as the quantity a programmer manipulates.

Remark 2.3 (Non-compositionality is not a defect). The store denotation of ρ is a monoid homomorphism for parallel composition, because it denotes the store. The sum-over-futures denotation cannot be, because composing P with Q creates races that neither had alone. This is not a weakness relative to quantum mechanics; it is the same phenomenon in the same place — interacting subsystems do not have product states. The cut is the interaction term, and entanglement across a cut is the semantic image of a race across a cut. It is also the semantic shadow of the observation that an aperture is a property of the cut and not of either agent flanking it.

3 What unmodified ρ already gives

3.1 The race equation

Take as the constraint on an interpretation $\llbracket - \rrbracket$, at a two-way race between one receive and two sends,

$$\llbracket x!(Q_1) \mid \text{for}(y \leftarrow x)P \mid x!(Q_2) \rrbracket = w_1 \llbracket P\{\text{@}Q_1/y\} \mid x!(Q_2) \rrbracket + w_2 \llbracket x!(Q_1) \mid P\{\text{@}Q_2/y\} \rrbracket, \quad (1)$$

with $w_1 = w_2$ and the weights covering the total probability space; dually for two receives racing over one message. The constraint says nothing about a lone receive meeting a lone send. Alongside it we adopt the principle that the meaning of a term is the sum of all its possible futures.

Two things are worth noticing immediately about (1). It is *parameter-free*: the weights are forced by the arity of the race, so there is no table, no key language, and no partition. And it is *silent on the confluent fragment*, which localises whatever design freedom remains.

3.2 The residual is a which-path record

The left summand of (1) retains $x!(Q_2)$ and the right retains $x!(Q_1)$. That surviving message records which branch was taken. Two derivations whose residuals differ do not reconverge, and derivations that do not reconverge cannot interfere.

There is exactly one escape, and it is forced: when $Q_1 \equiv Q_2$ the record is vacuous, the two summands are structurally congruent, and the branches meet.

Proposition 3.1 (Stage-one verdict). *Under (1) with uniform real weights and no further apparatus, the only interference available to ρ is between derivations consuming structurally congruent messages, and it is always constructive.*

Proof sketch. Interference requires two derivations from a common source reaching a common target. At a single race the targets are $P\{\text{@}Q_1/y\} \mid x!(Q_2)$ and $x!(Q_1) \mid P\{\text{@}Q_2/y\}$, which are congruent only if $Q_1 \equiv Q_2$ (the substitution instances agreeing for all P forces the quoted payloads to agree). Iterating, any reconvergence decomposes into steps of this form together with permutations of independent redexes, which are identified by Definition 4.1 below. All weights being non-negative reals, summed amplitudes cannot cancel. $\square$

Remark 3.2 (The general race is a permanent). At a channel carrying a bag of produces and a bag of consumes, communication chooses an admissible pairing and zips, leaving surplus. Summing coherently over perfect matchings of an amplitude matrix is the computation of a permanent, which is to say that the coherent reading of the zip is BosonSampling-shaped: plausibly hard to sample, while the classical reading — pick a matching uniformly — is easy. The superlinear multiplicity that earlier work identified in the degenerate case is accordingly not a bookkeeping error but genuine indistinguishable-particle statistics.

The verdict is therefore genuinely good news for the design, because it makes the brief precise: what rho lacks is not interference *per se* but *phase* and *control over recombination*. The analogy with linear optics is close enough to be useful. Indistinguishable bosons through a passive network give hardness without universality; adding phase control and adaptivity is what carries one to KLM-style universality. The gap to be closed may be small.

3.3 The normalisation fork

Equation (1) is not consistent as stated, and the inconsistency shows up at its own simplest instance. With $Q_1 \equiv Q_2$ the right-hand side is $(w_1 + w_2)\llbracket T \rrbracket$ for a single T . If the w_i are amplitudes with $|w_1|^2 + |w_2|^2 = 1$, this has norm $\sqrt{2}$; for an m -fold degenerate race, norm $\sqrt{m}$ and weight m .

Three resolutions are available.

1. *Convex*. Let $\llbracket - \rrbracket$ take values in formal convex combinations, $w_i = 1/m$. Consistent, and exactly the classical scheduler with a fair coin. No interference.
2. *Branch register*. Adjoin the derivation index so the summands stay distinguishable. Consistent and interference-capable, at the cost of carrying the index — this is the route taken by the ZX compilation work.
3. *Unnormalised*. Let $\llbracket - \rrbracket$ take values in the free $\mathbb{V}$ -module on normal forms and apply the Born rule at observation. Consistent, with m -fold degeneracy carrying weight m^2 .

We take the third, on the ground that linear-in- m and quadratic-in- m are the values of two different observables rather than two normalisations of one: the first is what one gets by looking at which derivation fired, the second by declining to look. Since the whole point of the exercise is to decline to look, the quadratic answer is the correct one here, and it is exactly the bosonic enhancement of Remark 3.2.

4 Two requirements, derived

Interference needs two things and nothing else: derivations that *reconverge*, and a nonzero *relative phase* between them. Every syntactic feature proposed below is answerable to one of these.

4.1 Reconvergence must be counted over causal histories

A caution first, because it disciplines the path sum. Every concurrency diamond is a reconvergence: two independent redexes fired in either order reach the same term. Rho programs are saturated with these. If the sum over futures ranges over *interleavings*, then m independent redexes contribute a factor of $m!$ arising from nothing but the number of linearisations of a partial order.

Definition 4.1 (Independence and causal histories). Two redex firings at a term are *independent* when their footprints — the data and continuations they consume — are disjoint. Let $\sim$ be the least congruence on derivations identifying $u; v$ with $v; u$ for independent u, v . A *causal history* is a $\sim$ -class of derivations, and the path sum of §6.3 ranges over causal histories.

Proposition 4.2 (No spurious combinatorial enhancement). *Summing over causal histories rather than interleavings removes the $m!$ factor, and does not remove any reconvergence between causally inequivalent derivations.*

The apparatus for this choice already exists in the history-monad account of rho: free histories form the universal cover of the reduction graph and the erasure lattice is indexed by subgroups of its fundamental group. Filling the concurrency squares is one such erasure, and interference lives on the loops that survive it.

4.2 The two syntactic requirements

- (R1) **Phase conditional on data.** The amplitude attaching to a branch must be allowed to depend on which datum that branch consumed, and on the structure of the term around it.
- (R2) **Which-path erasure.** The programmer must be able to arrange that two branches produce the *same* residual, so that their amplitudes are added rather than kept apart.

Remark 4.3 (Auditing weight maps against the requirements). A weight map answers (R1) well: its keys are spatial-behavioural formulae refining the left-hand side of a rule, which is precisely “a phase conditional on the structure of what was matched.” It says nothing whatever about (R2); in the ZX treatment, recombination arrives as a choice of measurement basis at the end, which is an observer’s decision made outside the program. And it is ergonomically misplaced: a table carried in the state is a Hamiltonian specification, whereas arranging interference is done with term formers, at the site where the branching happens.

5 The design

The design is one change, stated in a sentence: *the where clause of the for-comprehension is valued in a commutative semiring and is evaluated once per candidate match.*

5.1 Resolution algebras

Definition 5.1 (Resolution algebra). A *resolution algebra* is a commutative semiring $\mathbb{V} = (V, \oplus, \otimes, 0, 1)$: $\oplus$ associative and commutative with unit 0, $\otimes$ associative and commutative with unit 1, $\otimes$ distributing over $\oplus$, and 0 annihilating.

The two operations are forced by what the connectives must mean.

- *Conjunction is $\otimes$.* Two conditions holding together compose multiplicatively; this is also what makes the joined receive $p_1 \leftarrow c_1$ & $p_2 \leftarrow c_2$ combine its clauses correctly.
- *Disjunction is $\oplus$.* Two ways for a match to be enabled add.

Principle 5.2 (Interference is a logical phenomenon). Since $\llbracket \varphi \vee \psi \rrbracket = \llbracket \varphi \rrbracket \oplus \llbracket \psi \rrbracket$, in a resolution algebra with additive inverses a disjunction may evaluate to 0 though neither disjunct does. Destructive interference is a *false disjunction with true disjuncts*. This is the whole of the additional structure, stated in the logic rather than imposed on it from outside.

Negation is the casualty and it is worth saying so plainly. On $[0, 1]$ the map $v \mapsto 1 - v$ serves; on $\mathbb{C}$ there is no canonical complement and De Morgan fails against $(+, \times)$. The design therefore keeps two sorts, with the crisp sort embedded in the graded one.

Algebra	$\oplus, \otimes$	Resolver	Regime
$\mathbb{B} = (\{0, 1\}, \vee, \wedge)$	$\vee, \wedge$	demonic / RSpace race	rholang 1.4 exactly
$\mathbb{R}_{\geq 0} = (\mathbb{R}_{\geq 0}, +, \times)$	$+, \times$	Gillespie / SSA	stochastic
$\mathbb{C} = (\mathbb{C}, +, \times)$	$+, \times$	Born rule	interference
Viterbi $([0, 1], \max, \times)$	$\max, \times$	greedy	most-likely path
Tropical $(\mathbb{R} \cup \{\infty\}, \min, +)$	$\min, +$	least cost	cost-optimal path
pow-PLN $([0, 1], +, \times)$	$+, \times$	inference-driven	§10

Table 1: Instances. One syntax; the resolver is the choice of $\mathbb{V}$.

5.2 Two sorts of formula

Let the crisp sort be the existing condition language of rholang 1.4: ground formulae supplied by a language definition, the spatial connectives generated from the term formers, the context-labelled modality $\langle K \rangle$, and full Boolean structure.

$$\beta ::= \text{true} \mid \text{false} \mid g \mid \neg\beta \mid \beta \wedge \beta \mid \beta \vee \beta \mid \langle K \rangle \beta \mid \text{out}(n)\beta \mid \text{in}(n)\beta \mid \beta \mid \beta \mid @\beta$$

and let the graded sort be

$$\psi ::= [\beta] \mid v \mid \psi \otimes \psi \mid \psi \oplus \psi \mid \langle K \rangle_{\mathbb{V}} \psi \quad (v \in V).$$

Here $[\beta]$ is the indicator embedding, sending a satisfied β to 1 and an unsatisfied one to 0, and v ranges over scalar constants of the algebra. The graded sort has no negation and no fixed-point operator; the crisp sort keeps both.

Remark 5.3 (One slot, not two). It is tempting to give the for-comprehension two clauses, a Boolean gate and a graded weight. It is better to give it one graded clause and recover the gate as its *support*: a communication is enabled exactly when its clause evaluates to something other than 0. This keeps the language unforked — existing Boolean where-clauses are the $\mathbb{B}$ -valued fragment under $\mathbb{B} \hookrightarrow \mathbb{V}$ — and it makes support adequacy (Definition 2.2) hold by construction rather than by proof, since the classical interpreter sees nothing of the value except whether it vanishes.

5.3 Syntax

The for-comprehension of rholang 1.4 already carries a **where** clause, whose condition may mention the variables bound by the preceding receipts. The change is only to the sort of that condition.

```
for( p11 <- c11 & ... & pm1 <- cm1 ;
    ... ;
    pln <- cln & ... & pmn <- cmn
    where psi ) P
```

with **psi** a graded formula over the algebra $\mathbb{V}$ declared for the shard or the channel’s language definition. Within **psi**, the bound names p_{ij} are in scope, so the value may depend on what was matched — which is requirement (R1).

5.4 Per-match evaluation

The decisive point, and the one from which everything in §8 follows, concerns *when* the clause is evaluated.

Definition 5.4 (Candidates). Let r be a receipt site in P — a for-comprehension together with the channels its receipts name. A *candidate* at r is an assignment σ of a distinct datum occurrence to each pattern of each group, such that every pattern matches its assigned datum. Write $\text{Cand}(P, r)$ for the set of candidates and $\text{out}(r, \sigma) \in \mathcal{P}/\equiv$ for the residual term the firing of r under σ produces.

Candidates are indexed by *occurrences*, not by terms. This matters: two structurally congruent messages on a channel are two candidates, not one, because parallel composition is a multiset and structural congruence does not delete duplicates. An implementation that keys data by content alone, or that allocates identifiers during firing, cannot represent Cand faithfully.

Definition 5.5 (Graded clause valuation). A graded clause ψ at a receipt site r denotes a function

$$\llbracket \psi \rrbracket_r : \text{Cand}(P, r) \longrightarrow V,$$

defined by structural recursion, with $\llbracket \beta \rrbracket$ evaluated against the term P under the substitution σ carries, scalars constant, $\otimes$ and $\oplus$ pointwise, and $\langle K \rangle_{\mathbb{V}}$ as in §5.5.

So a clause is a *vector* indexed by candidates, and — once we compose it with out — a *matrix* over candidates and outcomes. That is the whole mechanism.

5.5 The graded modality is a transfer operator

Let $\mathbb{V}\langle \mathcal{P}/\equiv \rangle$ be the free $\mathbb{V}$ -module on terms modulo structural congruence. Define the one-step operator of a term by summing its candidates:

$$T(P) = \sum_r \sum_{\sigma \in \text{Cand}(P, r)} \llbracket \psi_r \rrbracket_r(\sigma) \cdot \text{out}(r, \sigma), \quad (2)$$

the outer sum over receipt sites, taken over causal histories in the sense of Definition 4.1, and the coefficient arithmetic in $\mathbb{V}$. Then the graded modality is

$$\llbracket \langle K \rangle_{\mathbb{V}} \psi \rrbracket(P) = \sum_{r, \sigma} \llbracket \psi_r \rrbracket_r(\sigma) \otimes \llbracket \psi \rrbracket(\text{out}(r, \sigma)),$$

which is T acting on the vector of values of ψ . In other words the graded modality is the transfer matrix of the reduction relation, and the principle that the meaning of a term is the sum of all its possible futures is exactly the statement that $\llbracket - \rrbracket$ is a fixed point of $\langle K \rangle_{\mathbb{V}}$. The denotational principle and the graded logic are one object seen twice.

6 Formal semantics

6.1 Terms

We work with core rho, taken with a built-in name restriction whose desugaring is not our concern here.

$$P ::= 0 \mid x!(P) \mid \text{for}(\vec{y} \Leftarrow \vec{x} \text{ where } \psi)P \mid P \mid P \mid *x \mid \text{new } x \text{ in } P, \quad x ::= @P.$$

Quoting is $@P$ and dropping is $*x$. Structural congruence $\equiv$ is the least congruence making $(\mathcal{P}, |, 0)$ a commutative monoid, together with the usual laws for **new** and $*@P \equiv P$.

6.2 The state space

Definition 6.1 (Graded denotation). Fix a resolution algebra $\mathbb{V}$ and a budget $\beta \in \mathbb{N}$. Let $M := \mathbb{V}\langle \mathcal{P}/\equiv \rangle$ be the free $\mathbb{V}$ -module on terms modulo structural congruence, with basis vectors written $|P\rangle$. The β -budgeted denotation is

$$\llbracket P \rrbracket_\beta := T_*^\beta |P\rangle = \sum_{h: P \Rightarrow^{\leq \beta} Q} w(h) |Q\rangle,$$

where T_* extends T of (2) linearly, h ranges over causal histories of length at most β ending at a normal form or exhausting the budget, and $w(h)$ is the $\otimes$ -product of the clause values along h .

Remark 6.2 (Why a budget, and only one). The definition is corecursive and does not bottom out on non-terminating terms. For terminating P the recursion is well-founded on the reduction tree and β may be taken to be its depth. In general one needs either a convergence hypothesis — a spectral radius condition on T , since $\mathbb{C}$ has no order and Knaster–Tarski is unavailable — or a budget. We take the budget, and note it is the same budget already charged elsewhere: the risk ledger’s single ω for reflection, the address-space bound of the compilation route, and the inference-token budget that keeps generated model-checking terminating. These should be one budget doing three jobs rather than three budgets.

To make normal forms absorbing and keep histories comparable in length we adopt the usual convention.

Definition 6.3 (Fictitious jump). At a normal form Q the operator acts as $T|Q\rangle = |Q\rangle$.

6.3 Readings

One denotation admits three readings, differing only in what is done at the end.

1. *Support*. $\text{supp}(\llbracket P \rrbracket_\beta) \subseteq \text{NF}$. This is what the classical interpreter sees, and by Remark 5.3 it is all it sees.
2. *Born*. For $\mathbb{V} = \mathbb{C}$, the distribution $Q \mapsto |\langle Q | \llbracket P \rrbracket_\beta \rangle|^2$, normalised.
3. *Stochastic*. For $\mathbb{V} = \mathbb{R}_{\geq 0}$, the same expression without the square, which is the jump-chain marginal of the Gillespie reading.

Theorem 6.4 (Conservativity). *Let $\mathbb{V} = \mathbb{B}$ and let every clause be the embedding $\lceil \beta \rceil$ of a crisp condition. Then $\text{supp}(\llbracket P \rrbracket_\beta)$ is exactly the set of normal forms reachable from P in at most β steps under rholang 1.4’s operational semantics with its Boolean *where* clauses.*

Proof sketch. In $\mathbb{B}$, $\otimes = \wedge$ and $\oplus = \vee$, so $w(h) = 1$ iff every step of h has a satisfied clause, and the coefficient of $|Q\rangle$ is the disjunction over histories reaching Q . Candidates coincide with the admissible zips of the store presentation, and the fictitious jump does not add normal forms. Hence the coefficient is 1 iff some admissible derivation reaches Q . $\square$

Corollary 6.5 (Support adequacy). *For any $\mathbb{V}$, $\text{supp}(\llbracket P \rrbracket_\beta) \subseteq \text{Reach}_\beta(P)$, since a nonzero coefficient requires at least one history with all clause values nonzero, and such a history is admissible under the Boolean image of the clauses. Equality fails exactly when some Reach-witnessed coefficient is a sum of nonzero terms summing to 0 — that is, exactly on the interference deficit $\mathcal{D}(P)$.*

This is the payoff of Remark 5.3. Adding grading cannot make a classical program do something it could not do; it can only make it fail to do something it could.

7 Weight maps are a normal form

Definition 7.1 (Weight map, recalled). A weight map for a rewrite rule R is a finite family of spatial-behavioural formulae $\chi_1, \dots, \chi_k$ refining the left-hand side of R , required to *partition* that refinement, together with values $v_1, \dots, v_k \in V$; the weight of a redex is v_i for the unique i with χ_i satisfied.

Theorem 7.2 (Weight maps are the linear graded clauses). *Every weight map $(\chi_i, v_i)_{i \leq k}$ is denoted by the graded clause*

$$\psi = \bigoplus_{i=1}^k v_i \otimes [\chi_i],$$

and conversely a graded clause of this shape denotes a weight map if and only if the χ_i partition the refinement. Graded clauses not of this shape — those with nested $\otimes$ over overlapping conditions, or with a graded modality — are strictly more expressive.

Proof sketch. Forwards: on a candidate satisfying χ_j and no other, all but the j th summand vanish and the value is v_j , and totality of the partition ensures some summand survives. Backwards: if two χ 's overlap on a candidate, the clause returns $v_i \oplus v_j$, which is a well-defined value but not a table lookup; the partition condition is exactly the condition making the sum a selection. Strictness: with overlapping χ_i the clause is a genuine mixture no weight map assigns, and $\langle K \rangle_V$ inspects successors, which no key can. $\square$

Corollary 7.3 (The partition discipline is unnecessary). *Single-valuedness of a graded clause is automatic, a formula having one value. The requirements that key sets partition the left-hand-side refinement, that partitions be constructed through a single checked funnel, and that a default class complete them, all exist to secure single-valuedness for a lookup and have no analogue here.*

This also dissolves a specific obstruction. Earlier work observed a tension between the *Boolean demand* — a weight map's partition needs complements, so the target logic specification must supply negation — and the fact that the transport of a generated logic along a bisimulation-preserving translation preserves only the geometric fragment, which has no complements. With no partition to define, the Boolean demand does not arise. Negation is still absent from the graded sort, but it is absent as a design choice about the value algebra rather than as an unmet requirement.

Proposition 7.4 (Plasticity is free). *A dynamic weight map, whose table is revised after each rewrite by an update function $u : W \times \text{Match} \times \text{Trace} \rightarrow W$, is denoted by a static graded clause whenever u 's dependence on the trace is expressible as a crisp formula over the current term. No update function is needed, because a clause is re-evaluated against the current term at every candidate.*

Proof sketch. The clause ψ mentions the term through its crisp subformulae; its value at a candidate in the term P' reached after some history is by definition computed against P' . A table revised to depend on features of P' therefore agrees with the clause $\bigoplus_i v_i \otimes [\chi_i]$ in which the χ_i mention those features directly. Dependence on trace history beyond what is visible in P' is not expressible, which is the hypothesis. $\square$

Remark 7.5. Proposition 7.4 retires a specific implementation obligation. The plan for compiling the weighted simulator identified the update function — an arbitrary closure over time and a trace summary — as the one component with no compilable form, and proposed a declarative sub-language for it. The graded clause *is* that sub-language, and it was already needed for other reasons. The restriction to features visible in the current term has precedent: the exhaustive-graph construction already refuses time-dependent updates by fixing the clock at zero.

8 The join is the recombiner

Requirement (R2) asked for which-path erasure, and the audit of Remark 4.3 found nothing in the weight map that supplies it. It turns out no new term former is needed: the erasure is already in the language, in the join.

8.1 Permutations of a join are the interfering pair

Consider a receipt whose group contains m patterns on a single contended channel, meeting m data occurrences:

$$x!(Q_1) \mid \cdots \mid x!(Q_m) \mid \text{for}(y_1 \Leftarrow x \ \& \ \cdots \ \& \ y_m \Leftarrow x \ \text{where } \psi)P.$$

The candidates are the $m!$ bijections σ from patterns to data. Every candidate consumes *all* the data — there is no surplus, hence no residual message and no which-path record — and produces $\text{out}(\sigma) = P\{\text{@}Q_{\sigma(1)}/y_1, \dots, \text{@}Q_{\sigma(m)}/y_m\}$.

Definition 8.1 (Symmetric continuation). P is *symmetric* in $y_1, \dots, y_m$ when $P\{\text{@}Q_{\sigma(1)}/y_1, \dots\} \equiv P\{\text{@}Q_{\tau(1)}/y_1, \dots\}$ for all bijections σ, τ and all payloads.

Symmetry is cheap to arrange, because parallel composition is commutative: the continuation $z!(\ast y_1) \mid z!(\ast y_2)$ is symmetric, while $z_1!(\ast y_1) \mid z_2!(\ast y_2)$ is not.

Theorem 8.2 (The clause is a scattering matrix). Let $A_{ij} := \llbracket \psi \rrbracket(y_i \mapsto \text{@}Q_j)$ be the value of the clause on the candidate assigning datum j to pattern i , and suppose ψ factors as $\bigotimes_i \psi_i(y_i)$ over patterns. If P is symmetric in the y_i , all $m!$ candidates share a single outcome $\bar{P}$, and

$$\langle \bar{P} \mid T \mid P\text{-site} \rangle = \sum_{\sigma \in S_m} \prod_{i=1}^m A_{i\sigma(i)} = \text{Per}(A).$$

If P is not symmetric, the candidates have pairwise distinct outcomes and each carries its own product with no summation.

Proof sketch. Every candidate is a bijection and the clause factors, so the weight of the candidate σ is $\prod_i A_{i\sigma(i)}$. Under symmetry all outcomes are one basis vector and the coefficients add, which is the permanent. Without symmetry the outcomes are distinct basis vectors and no addition occurs. $\square$

Corollary 8.3 (Which-path, syntactically). *No choice of clause produces interference at a receipt whose continuation distinguishes its bound names. Whether a race interferes is decided by the continuation, not by the weights.*

This is the design’s central ergonomic fact. The programmer has two dials at exactly the site where the branching happens: the clause sets the amplitudes, and the continuation decides whether the branches are told apart. Arranging interference is arranging for the continuation to forget.

Remark 8.4 (Statistics). $\text{Per}(A)$ is the bosonic amplitude. A clause assigning the sign of σ — expressible when the algebra has additive inverses and the crisp sort can detect the relevant order — gives $\text{Det}(A)$ and hence fermionic statistics, in which coincident data annihilate. That the same syntactic slot produces both is a point in favour of the slot.

8.2 The minimal interferometer

Take $\mathbb{V} = \mathbb{C}$, $m = 2$, and a clause that returns a on the identity assignment and b on the transposition. With a symmetric continuation:

```
new x, z in {
  x!(Q1) | x!(Q2) |
  for( y1 <- x & y2 <- x where psi ) { z!(*y1) | z!(*y2) }
}
```

The single outcome $z!(*Q_1) | z!(*Q_2)$ carries amplitude $a + b$. Choosing $b = -a$ makes it zero. That normal form is operationally reachable — indeed it is the *only* reachable normal form — so $\mathcal{D}(P) \neq \emptyset$ and Corollary 6.5’s inclusion is strict. Replacing the continuation by $z_1!(*y_1) | z_2!(*y_2)$ restores two distinct outcomes with amplitudes a and b , probabilities $|a|^2$ and $|b|^2$, and no cancellation whatever the phases.

Three lines of rholang thus exhibit the entire phenomenon: a detector that never fires, and a which-path measurement that destroys the effect. It is the Hong–Ou–Mandel arrangement, with the channel z playing the role of a single detector and the pair z_1, z_2 of two.

Remark 8.5 (What a rho context can and cannot do). Note the consistency with §2. The context $\text{for}(y_1 \leftarrow x \ \& \ y_2 \leftarrow x \dots) \{z!(*y_1) | z!(*y_2)\}$ does not observe the resolution of the race; it *arranges* the race so that resolution leaves a trace in the support. This is precisely what was meant by a context acting as an interferometer, and it is the reason the enterprise has observable consequences without any context ever reading an amplitude.

9 A worked example: Shor’s algorithm

The design of §5 was derived from what interference requires, not from the circuit model, so it is a fair question whether it reaches the standard algorithms. It does. The instructive part is not that the fragment turns out expressive enough but that it turns out *too* expressive, which §9.5 addresses and which changes the status of one of this note’s open threads.

Every number reported in this section is computed by a simulator implementing Definition 6.1 literally: it enumerates the candidates at each site and adds coefficients on coinciding outcomes. Nothing in it multiplies gate matrices. Matrices appear only as the values a clause returns on a candidate.

9.1 A qubit and a gate

Definition 9.1 (Wire encoding). A *qubit* is a channel carrying exactly one of two designated data. Writing a_0, a_1 for the two, the basis states are $q!(a_0)$ and $q!(a_1)$.

```
new q, w, q' in {
  q!(*in) | w!(a0) | w!(a1) |
  for( v <- q & u1 <- w & u2 <- w where psi ) { q'!(*u1) }
}
```

Lemma 9.2 (Arbitrary one-qubit gate). *The site above has exactly two candidates, the bijections $\{u_1, u_2\} \rightarrow \{a_0, a_1\}$. Each consumes every datum on q and on w , so no residual records which fired. The outcomes are $q'!(a_0)$ and $q'!(a_1)$; they are distinct, and neither mentions v , hence they are the same two terms for either input encoding. Therefore the one-step operator restricted to $\text{span}\{q!(a_0), q!(a_1)\}$ is the matrix $A_{ij} = \llbracket \psi \rrbracket(v \mapsto a_i, u_1 \mapsto a_j)$, and since ψ may mention both v and u_1 , every 2×2 matrix over $\mathbb{V}$ is realised.*

Remark 9.3 (Two erasures, not one). Theorem 8.2 identified outcomes of *different candidates at one site*, which is recombination within a superposition. A gate needs the other identification: outcomes at *different input basis states* must coincide, which is what makes the site a linear map into a fixed codomain rather than a family of unrelated branchings. Both are bought by the same device — a join consumes everything, and the continuation re-emits without mentioning what was consumed. That the join’s totality purchases both erasures is the reason no further term former is needed here either.

Lemma 9.4 (Arbitrary two-qubit gate). *Joining two input channels with two ancilla channels gives $2 \times 2 = 4$ candidates and four distinct outcomes indexed by the pair of u_1 -assignments. The clause may mention both input data and both assignments, so every 4×4 matrix is realised, including entangling ones. Diagonal gates are the case where the clause vanishes off the diagonal, and by Remark 5.3 those candidates simply do not fire.*

9.2 Composition, and where the circuit’s wires went

Proposition 9.5 (The circuit’s DAG is the term’s causal order). *A gadget is enabled exactly when its input channels carry data. Composing gadgets in series on fresh channels therefore makes the data dependency of the term equal to the wire structure of the circuit, and every reduction order is a linear extension of the circuit’s dependency DAG. No scheduling discipline is required and none is expressible: the term does not encode the circuit’s order, it is that order.*

Proposition 4.2 is load-bearing here rather than hygienic. Gates in one circuit layer are independent redexes, so without the causal quotient a layer of n gates would contribute a factor $n!$ from counting linearisations. Measured, for n independent one-qubit gadgets: the causal sum gives squared norm 1.000000 for $n = 2, 3, 4$, while summing over interleavings inflates it by exactly 2, 6 and 24.

9.3 The classical part is free

Proposition 9.6 (Deterministic communication is amplitude-transparent). *A communication with a single candidate contributes the clause’s value on that candidate, which for a crisp clause is 1. Hence amplitudes are unchanged by deterministic reduction, and the length of a derivation does not affect its weight.*

Corollary 9.7. *Any ordinary rho program runs inside every branch of a superposition at no cost in the semantics. In particular modular exponentiation — the expensive part of Shor’s algorithm, and the part usually requiring laborious reversible compilation — is written as ordinary rholang.*

Measured: the block computing $x \mapsto 7^x \bmod 15$ carries a 16-term superposition to a 16-term superposition with norm 1.000000000000, exactly preserved.

9.4 The instance, assembled

Theorem 9.8 (Expressibility). *For N with $n = \lceil \log_2 N \rceil$, Shor’s algorithm is a graded rho term using $O(n^2)$ gadgets of arity at most two for the quantum Fourier transform, $O(n^3)$ deterministic communications for modular exponentiation, and a term size and reduction depth polynomial in n .*

Two verifications.

The transform. The QFT assembled from the gadgets of Lemmas 9.2–9.4 — Hadamards, controlled phases and swaps, in the standard decomposition — was compared entry by entry

against the discrete Fourier matrix $\omega^{jk}/\sqrt{2^n}$. Worst entry error: 2.7×10^{-16} at $n = 2$, 1.4×10^{-15} at $n = 3$, 3.8×10^{-15} at $n = 4$.

The instance. $N = 15$, $a = 7$ (so $r = 4$), with four counting and four work qubits. The run fires 8 one-qubit and 8 two-qubit gadgets, evaluating 390 candidates in all, and ends with 16 configurations carrying nonzero amplitude. The counting register reads

outcome c	0	4	8	12	all others
probability	0.250000000	0.250000000	0.250000000	0.250000000	0

Table 2: Counting-register distribution, total 1.0000000000000.

Continued fractions on $4/16$ and $12/16$ give $r = 4$; then $\gcd(7^2 - 1, 15) = 3$ and $\gcd(7^2 + 1, 15) = 5$.

The zeroes in Table 2 are the point. Twelve of the sixteen outcomes are operationally reachable — some derivation of the term produces each of them — and carry amplitude zero. By Definition 2.2 they constitute the interference deficit $\mathcal{D}(P)$, and by Corollary 6.5 the inclusion $\text{supp}(\llbracket P \rrbracket) \subseteq \text{Reach}(P)$ is strict by exactly those twelve. Shor’s algorithm, in this semantics, *is* a large interference deficit arranged on purpose.

9.5 The hazard: unnormalised clauses buy post-selection

§3.3 chose the unnormalised reading, with the Born rule applied at observation. That was the right call for the degenerate race, but nothing in §§5–6 requires a clause matrix to be an isometry. A non-isometric clause shrinks the norm, and renormalising at observation is precisely post-selection on the surviving branch.

Proposition 9.9 (Overshoot). *With $\mathbb{V} = \mathbb{C}$ and arbitrary clauses, the graded fragment with the Born reading decides PostBQP, and hence PP.*

Proof sketch. Lemmas 9.2–9.4 give universal quantum circuits. The clause $\text{diag}(1, \epsilon)$ is expressible and non-isometric; applying it and normalising at observation implements post-selection on the first outcome up to error $O(\epsilon)$. Conversely the model is simulable with post-selection. Aaronson’s identification of PostBQP with PP gives the second equality. $\square$

Measured, to show that the semantics charges nothing for it: a Hadamard layer on n qubits followed by $\text{diag}(1, 10^{-3})$ on each wire leaves a surviving norm of 2.5×10^{-1} , 1.25×10^{-1} , 6.25×10^{-2} , 3.13×10^{-2} for $n = 4, 6, 8, 10$ — falling as $2^{-n/2}$ — while the renormalised probability of the all-zero string is 0.999996, 0.999994, 0.999992, 0.999990. The filter is free.

So the fragment as specified in §6 is not a quantum programming language. It is a post-selected one, and it computes PP — far more than Shor, and far more than any device. The containment condition must be stated.

Definition 9.10 (Isometric clause). Let r be a receipt site, let I index the input configurations on which r is enabled, and let O index the outcomes. The clause ψ at r is *isometric* when the matrix $A_{io} := \sum_{\sigma \in \text{Cand: out}(\sigma)=o} \llbracket \psi \rrbracket(\sigma)$, taken at input i , satisfies $A^\dagger A = \mathbb{I}$.

Proposition 9.11 (Decidability of the condition). *For a clause of the normal form of Theorem 7.2 with constant $v_i \in V$ and crisp χ_i , the matrix A is a finite table computable from the clause and the site’s arity, and isometry is decidable by inspection. For clauses using the graded modality $\langle K \rangle_{\mathbb{V}}$ the entries depend on the reachable successors and no such decision procedure is immediate.*

This gives a natural stratification, and it is the same one extraction wants: *the modality-free fragment with isometric clauses is the extractable fragment*, and it is the fragment a compiler to circuits can consume. Clauses using $\langle K \rangle_{\forall}$ remain available for the stochastic and inference-driven regimes of §10, where no unitarity is expected of them.

9.6 What the example shows, and what it does not

It shows that expressiveness is not the obstacle: the fragment is universal for quantum circuits, and the classical portion of a quantum algorithm is written as ordinary rholang.

It does not show that graded rho has a quantum idiom of its own. Every gate above is a hand-built join gadget, and mobility, reflection and name-passing sit idle throughout; the encoding is circuit-shaped, and a circuit-shaped encoding demonstrates capability rather than fit. The one primitive the design offers that a circuit does not hand over directly is Theorem 8.2: an m -fold join is a BosonSampling instance in a single term former. Since candidate sets are permutation sets, the ambient symmetry group of the design is S_m rather than $\mathbb{Z}_N$, which suggests that a native algorithm — if there is one — would sit nearer the non-abelian hidden subgroup problem than near Shor. We record the direction without claiming it.

Finally, writing Shor is not running Shor. Evaluating the denotation classically is exponential; the simulator used for this section is a state-vector simulator wearing a process calculus. The value of the term is as a *compilation source*, and Proposition 9.11 says exactly which terms a compiler can accept.

10 PLN-valued where-clauses

10.1 Why this slot invites PLN

MeTTaIL describes finitely presentable graph-structured lambda theories, and the control-flow layer is a rholang whose channels are typed by those language definitions. Probabilistic Logic Networks are the uncertain-inference framework developed for the OpenCog and Hyperon line of work, in which knowledge is annotated with truth values and inference rules carry truth-value formulas computing a conclusion’s annotation from its premises’.

The proposal of this note opens a door between them. If a **where** clause is valued in an algebra rather than in the Booleans, then a clause may be valued *by an inference*: the propensity with which a communication fires is the truth value PLN currently assigns to the proposition that it should. Execution stops being gated by belief and starts being *driven* by it. The scheduler and the reasoner become the same loop.

This is a different use of the machinery from the quantum one, and it is worth being explicit that it is a different *regime* rather than a different mechanism.

10.2 PLN truth values, briefly

We use PLN’s *simple truth values*: a pair $(s, c) \in [0, 1]^2$ of a strength, the estimated probability that a statement holds, and a confidence, reflecting how much evidence stands behind that estimate.¹ The independence-based deduction formula for term-logic transitivity is

$$s_{AC} = s_{AB}s_{BC} + \frac{(1 - s_{AB})(s_C - s_B s_{BC})}{1 - s_B},$$

with a concept-geometry variant replacing the independence assumption by an assumption about the shape of concepts, $s_{AC} = s_{AB}s_{BC} / \min(1, s_{AB} + s_{BC})$. Induction and abduction are

¹Confidence is formalised in several ways across the literature, including as the width of a confidence interval around the mean of a second-order distribution; the NARS-style count form $c = n/(n + k)$ for an evidence count n and a personality parameter k is the one easiest to read operationally, and we use it informally.

obtained by chaining deduction with Bayes' rule. The quantity $\text{pow} := s \times c$, sometimes called the power of an uncertain statement, behaves multiplicatively under deduction in the regime where term probabilities are highly uncertain: $\text{pow}_{AC} \approx \text{pow}_{AB} \text{pow}_{BC}$.

10.3 The resolution algebra, honestly

The design of §5 requires a commutative semiring. PLN's operations do not straightforwardly furnish one, and it is better to say where the difficulty lies than to paper over it.

Multiplication is unproblematic. Conjunction of independent statements multiplies strengths, and confidences compose multiplicatively under the usual independence assumption, so $\otimes$ may be taken componentwise; under the high-uncertainty regime this agrees with deduction on the power.

Addition is the problem. PLN's operation for combining two estimates of the same proposition is *revision*, a confidence-weighted average

$$s = \frac{s_1 c_1 + s_2 c_2}{c_1 + c_2},$$

with an accompanying rule for the revised confidence. Revision is not the addition of a semiring: it is idempotent on equal arguments, it does not have $(0, \cdot)$ as a two-sided unit in the required sense, and it does not distribute over componentwise multiplication. Taking $\oplus$ to be revision therefore makes the one-step operator (2) sensitive to the order in which candidates are aggregated, which is precisely the order the calculus refuses to fix.

Two responses are available, and we recommend the first.

1. **PLN supplies values; $\mathbb{R}_{\geq 0}$ supplies the algebra.** Let the graded sort contain ground formulae whose evaluation is a PLN inference, and let the clause's value be the resulting power $\text{pow} = s \times c \in [0, 1]$, with $\oplus = +$ and $\otimes = \times$ over $\mathbb{R}_{\geq 0}$. Then $\mathbb{V} = \mathbb{R}_{\geq 0}$, the semiring laws hold, and the whole apparatus of §§5–6 applies unchanged. Deduction's multiplicativity on the power means that chaining inference and chaining communication agree, which is the coherence one wants. The confidence coordinate is not discarded: it enters the propensity, so a poorly-evidenced belief yields a rarely-taken branch rather than a forbidden one.
2. **Carry (s, c) and weaken the algebra.** Keep the pair as the carrier and accept a lax structure in which aggregation is revision. This is more faithful to PLN but requires a canonical aggregation order, which must then be justified as not observable — a substantial obligation, and one that reintroduces exactly the kind of implementation-visible choice the design was meant to abstract.

Principle 10.1 (Fuzzy inference drives execution). Under the first response, a MeTTaIL rewrite fires with propensity equal to the power PLN currently assigns to the proposition, expressed as a graded ground formula in the rule's **where** clause, that the rewrite is warranted here. Because clauses are re-evaluated at every candidate against the current term (Proposition 7.4), revisions of belief take effect on the next step without any separate update mechanism.

10.4 Three regimes, one syntax

The point of Table 1 is now visible. The same slot, with the same formula language and the same per-match evaluation, gives:

- $\mathbb{B}$ — rholang 1.4, unchanged, by Theorem 6.4;
- $\mathbb{R}_{\geq 0}$ — stochastic execution, with PLN as a distinguished source of values, in which inference schedules computation;

- $\mathbb{C}$ — coherent execution with interference, in which the clause is a scattering matrix by Theorem 8.2.

It is worth stating the boundary between the second and third plainly, since it is easy to blur. PLN values live in $[0, 1]$ and carry no phase; PLN-driven execution is therefore stochastic and exhibits no cancellation. The interference regime requires additive inverses in the value algebra, which is a property of $\mathbb{C}$ and not of any probability-valued logic. What the two share is the mechanism; what separates them is whether $\oplus$ can cancel.

Remark 10.2 (A speculation worth naming, and not indulging). PLN has been given paraconsistent formulations, and paraconsistent logics are one of the places complex or otherwise non-order-theoretic truth values have been contemplated. Whether a paraconsistent PLN would furnish a value algebra with cancellation — and hence let inference and interference share not merely a slot but a semantics — is a real question and not one this note attempts.

11 Implementation

The design is deliberately small on the implementation side, and most of what it needs already exists.

Candidate enumeration. The store presentation gives the redex enumerator directly: the map from channels to rows of data and waiting continuations, together with the join index, is precisely `Cand`. The one requirement the current representation does not meet is occurrence identity — per-channel occupancy counts collapse the distinction between congruent data, and identifiers allocated during firing are not stable across branches. Candidates must be addressed by (channel, canonical slot), deterministically and branch-independently. This is the same addressing obligation that arises in the compilation route, and it should be solved once.

Basis identity. Interference is addition of coefficients on a common basis vector, so the implementation needs a canonical name for a term modulo structural congruence. The content-addressed store already supplies one. This also repairs a known defect: a configuration fingerprint that formats values to fixed decimal precision cannot distinguish 1, -1 and i , and therefore cannot represent the very distinctions the grading exists to make.

Clause evaluation. The crisp sort is the existing condition language and its evaluator; the graded sort adds scalars, two binary operations and the graded modality. The modality is the only construct requiring successor exploration, and it takes the inference budget.

Two readings out. A simulator should emit both the support/Born reading and the coherent sum, since by Corollary 6.5 they differ exactly on the interference deficit. If they never differ on a corpus of test theories, the grading is doing nothing and the implementation is wrong.

What retires. Partition construction and its checked funnel; the requirement that keys be structural, as a precondition for partition locality; the update-function plumbing and the declarative update sub-language it wanted; and the class-global phase, which was in any case observationally inert.

12 Open threads

Obligation 12.1 (Length-asymmetric reconvergence). Theorem 8.2 secures interference at a single join. The general question is which closed loops survive the causal quotient of Definition 4.1 in pure rho — that is, whether two causally inequivalent derivations of different lengths can reach a common term. Reflection is the natural suspect, since dropping a quoted process makes the residual communication count depend on which datum won. A negative answer would confine interference to joins; a positive one would open a second, non-local source.

Obligation 12.2 (Normalisation). §3.3 recommends the unnormalised reading with Born at observation. The alternative branches should be worked out far enough to confirm that the choice is a choice of observable rather than of correctness.

Obligation 12.3 (Enforcing isometry). *Upgraded by §9.5 from an open thread to a containment requirement.* Definition 9.10 states the condition and Proposition 9.11 says it is decidable on the modality-free fragment. What remains is to turn that into a static discipline — a judgement on clauses, checked at elaboration, refusing non-isometric clauses in a declared coherent mode while leaving them available in the stochastic and inference-driven modes where no unitarity is expected. Until that exists, a graded program should be read as post-selected.

Obligation 12.4 (Adaptivity and universality). A continuation may install further joins depending on what it received, which is adaptivity. Whether adaptive graded joins reach a universal fragment — the question KLM answers affirmatively for linear optics — is the question on which a verdict about advantage ultimately rests.

Obligation 12.5 (Categorical status). The construction looks like a Kleisli category for the free-module monad of $\mathbb{V}$ over terms modulo congruence, with the graded modality as the structure map of a coalgebra. Making this precise would connect the design to the existing coalgebraic account of the store denotation and its two-layer full abstraction.

Obligation 12.6 (Paraconsistent values). Remark 10.2: whether an inference calculus can supply a value algebra with cancellation.

13 Conclusion

The rho calculus does not say how its races are settled, and that silence is a parameter rather than a gap. Filling it differently gives different executions of the same program, and a programmer who wants to arrange interference needs to be able to write the filling down.

The change proposed here is one line of grammar: let the **where** clause of the for-comprehension take values in a commutative semiring, and evaluate it once per candidate match rather than once per communication. Boolean values give back today’s language exactly. Non-negative reals give stochastic execution, with probabilistic-logic inference as a natural source of values, so that what the system believes decides what it does next. Complex values give interference, and the clause becomes a scattering matrix whose permanent is the amplitude of the symmetric outcome.

Shor’s algorithm fits, and fitting it was informative in an unexpected direction. Universality is easy, the classical portion of the algorithm is ordinary rholang, and the algorithm’s characteristic behaviour shows up in this semantics as a large interference deficit arranged on purpose — twelve of sixteen reachable outcomes carrying amplitude zero. What the exercise exposed is that the fragment as first specified reaches too far: with clause matrices unconstrained, the renormalisation performed at observation is post-selection, and the fragment computes PP. The isometry condition is therefore not a compilation convenience but the boundary between

a quantum language and a magic one, and the fragment on which it can be checked — clauses without the graded modality — is precisely the fragment a circuit compiler can accept.

Two things about this were not evident at the outset. The first is that weight maps were not wrong but merely linear: they are the normal forms $\bigoplus_i v_i \otimes [\chi_i]$, and the partition discipline that attended them was the price of making a lookup single-valued rather than a fact about weights. The second is that the recombination interference requires — the half of the problem no weight map addressed — was already in the language. A join over a contended channel has one candidate per assignment and one outcome per orbit of the continuation’s symmetry, so whether a race interferes is decided by whether the continuation troubles to tell its inputs apart. That is a dial a programmer already knows how to turn.